

Q)1 Introduction to Database Creation

Objective: Set up and create a database using MySQL/PostgreSQL. Task: Entities and Relationships: • Student (StudentID, Name, Email, Age, Address) • Course (CourseID, CourseName, Credits, InstructorID) • Instructor (InstructorID, Name, Email, Department) • Enrollment (EnrollmentID, StudentID, CourseID, EnrollmentDate) Relationships: • A student can enroll in multiple courses. • A course can have multiple students. • An instructor teaches one or more courses.

```
CREATE DATABASE IF NOT EXISTS CollegeDB;
```

```
USE CollegeDB;
```

```
DROP TABLE IF EXISTS Enrollment;
```

```
DROP TABLE IF EXISTS Course;
```

```
DROP TABLE IF EXISTS Instructor;
```

```
DROP TABLE IF EXISTS Student;
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Age INT,  
    Address VARCHAR(100)  
);
```

```
CREATE TABLE Instructor (  
    InstructorID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Department VARCHAR(50)  
);
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT,  
    InstructorID INT,  
    FOREIGN KEY (InstructorID)  
    REFERENCES Instructor(InstructorID)
```

);

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    EnrollmentDate DATE,  
    FOREIGN KEY (StudentID)  
    REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID)  
    REFERENCES Course(CourseID)
```

);

```
INSERT INTO Student VALUES
```

```
(1, 'Arjun', 'arjun@gmail.com', 21, 'Pune'),  
(2, 'Rahul', 'rahul@gmail.com', 22, 'Mumbai'),  
(3, 'Sneha', 'sneha@gmail.com', 20, 'Delhi');
```

```
INSERT INTO Instructor VALUES
```

```
(101, 'Dr. Sharma', 'sharma@gmail.com', 'Computer Science'),  
(102, 'Dr. Mehta', 'mehta@gmail.com', 'Information Technology');
```

```
INSERT INTO Course VALUES
```

```
(201, 'Database Systems', 4, 101),  
(202, 'Python Programming', 3, 102),  
(203, 'Artificial Intelligence', 4, 101);
```

```
INSERT INTO Enrollment VALUES
```

```
(1, 1, 201, '2026-01-10'),  
(2, 2, 202, '2026-01-11'),  
(3, 3, 203, '2026-01-12'), (4,  
1, 202, '2026-01-13');
```

```
SHOW TABLES;
```

```
DESC Student;
```

```
SELECT * FROM Student;
```

```
SELECT * FROM Instructor;
```

```
SELECT * FROM Course;
```

```
SELECT * FROM Enrollment;
```

Q2) Views and Indexing

Optimizing a Student Course Management System with Views and Indexing for Faster Query Performance.

Objective: Understand views and indexing for performance optimization.

Task:

- Create views for easy reporting on students and course details.
- Update and delete records using views where applicable.
- Use indexing to speed up searches and measure query performance improvements.

```
CREATE DATABASE StudentManagement;
```

```
USE StudentManagement;
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100),  
    Age INT  
);
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50),  
    Credits INT  
);
```

```
INSERT INTO Student VALUES
```

```
(1, 'Arjun', 'arjun@gmail.com', 21), (2,  
'Rahul', 'rahul@gmail.com', 22),  
(3, 'Sneha', 'sneha@gmail.com', 20);
```

```
INSERT INTO Course VALUES
```

```
(101, 'Database Systems', 4),  
(102, 'Python Programming', 3),  
(103, 'Machine Learning', 4);
```

```
CREATE VIEW StudentView AS
```

```

SELECT StudentID, Name, Email, Age
FROM Student;

SELECT * FROM StudentView;

CREATE VIEW CourseView AS
SELECT CourseID, CourseName, Credits FROM Course;

SELECT * FROM CourseView;

UPDATE StudentView
SET Age = 23
WHERE StudentID = 1;

SELECT * FROM StudentView;

DELETE FROM StudentView
WHERE StudentID = 3;

SELECT * FROM StudentView;

CREATE INDEX idx_student_name
ON Student(Name);

CREATE INDEX idx_course_name
ON Course(CourseName);

EXPLAIN SELECT * FROM Student
WHERE Name = 'Arjun';

EXPLAIN SELECT * FROM Course
WHERE CourseName = 'Database Systems';

```

Q3) SQL Queries with Filters and Sorting ‘

Advanced Querying in a Student Course Management System Using Filters, Sorting, and Nested Subqueries Objective: Use SQL filters and sorting clauses. Task: • Filtering Data with WHERE (Find students above a certain age, filter by instructor) • Sorting with ORDER BY (Alphabetically, by credits) • Aggregation with GROUP BY (Student count per course, average age) • Nested Subqueries for Complex Queries

```

CREATE DATABASE StudentCourseDB;

USE StudentCourseDB;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,

```

```
Name VARCHAR(50),
Age INT,
Email VARCHAR(100)
);

CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50)
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    Credits INT,
    InstructorID INT,
    FOREIGN KEY (InstructorID)
    REFERENCES Instructor(InstructorID)
);

CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID)
    REFERENCES Course(CourseID)
);

INSERT INTO Student VALUES
(1, 'Arjun', 21, 'arjun@gmail.com'), (2,
'Rahul', 24, 'rahul@gmail.com'),
(3, 'Sneha', 20, 'sneha@gmail.com'),
(4, 'Priya', 23, 'priya@gmail.com');
```

```

INSERT INTO Instructor VALUES (101,
'Dr Sharma', 'Computer'),
(102, 'Dr Mehta', 'IT');
INSERT INTO Course VALUES
(201, 'Database', 4, 101),
(202, 'Python', 3, 102),
(203, 'AI', 5, 101);
INSERT INTO Enrollment VALUES
(1, 1, 201),
(2, 2, 201),
(3, 3, 202),
(4, 4, 203);
SELECT * FROM Student
WHERE Age > 21;
SELECT * FROM Course
WHERE InstructorID = 101;
SELECT * FROM Student
ORDER BY Name ASC;
SELECT * FROM Course
ORDER BY Credits DESC;
SELECT CourseID, COUNT(StudentID) AS TotalStudents
FROM Enrollment
GROUP BY CourseID;
SELECT AVG(Age) AS AverageAge
FROM Student;
SELECT Name
FROM Student
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollment
    WHERE CourseID = (
        SELECT CourseID

```

```

        FROM Course
        WHERE CourseName = 'Database'
    )
);

SELECT CourseName, Credits
FROM Course
WHERE Credits > (
    SELECT AVG(Credits)
    FROM Course
);

```

Q4) Multi-Table Joins

Develop a Customer Order Management System to Retrieve Data Using SQL Joins Objective: Learn inner and outer joins in SQL. Task: • INNER JOIN: Retrieve all customer orders (only customers who have placed orders). • LEFT JOIN: Retrieve all customers, including those who haven't placed any orders. • RIGHT JOIN: Retrieve all orders, including those placed by customers not in the Customers table. • FULL OUTER JOIN: Retrieve a complete list of customers and orders, including those that do not have a match in the other table.

```

CREATE DATABASE CustomerOrderDB;

USE CustomerOrderDB;

CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    City VARCHAR(50)
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    Product VARCHAR(50),
    Amount INT
);

INSERT INTO Customers VALUES
(1, 'Arjun', 'Pune'),

```

```
(2, 'Rahul', 'Mumbai'),  
(3, 'Sneha', 'Delhi'),  
(4, 'Priya', 'Chennai');
```

```
-- Insert Records into Orders INSERT
```

```
INTO Orders VALUES
```

```
(101, 1, 'Laptop', 50000),  
(102, 2, 'Mobile', 20000),  
(103, 1, 'Keyboard', 2000),  
(104, 5, 'Mouse', 1000);
```

```
SELECT Customers.CustomerID,
```

```
Customers.CustomerName,
```

```
Orders.OrderID,
```

```
Orders.Product
```

```
FROM Customers
```

```
INNER JOIN Orders
```

```
ON Customers.CustomerID = Orders.CustomerID;
```

```
SELECT Customers.CustomerID,
```

```
Customers.CustomerName,
```

```
Orders.OrderID,
```

```
Orders.Product
```

```
FROM Customers
```

```
LEFT JOIN Orders
```

```
ON Customers.CustomerID = Orders.CustomerID;
```

```
SELECT Customers.CustomerID,
```

```
Customers.CustomerName,
```

```
Orders.OrderID,
```

```
Orders.Product
```

```
FROM Customers
```

```
RIGHT JOIN Orders
```

```
ON Customers.CustomerID = Orders.CustomerID;
```

```
SELECT Customers.CustomerID,
```



```

Customers.CustomerName,
Orders.OrderID,
Orders.Product
FROM Customers
LEFT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
SELECT Customers.CustomerID,
Customers.CustomerName,
Orders.OrderID,
Orders.Product
FROM Customers
RIGHT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;

```

Q5) PL/SQL

Procedures and Functions Design and Implement a Sales Management System with Stored Procedures and Functions for Efficient Data Handling Objective: Implement stored procedures and functions. Task: • Create a stored procedure to add data to a table. • Create a function to calculate and return total sales.

```

CREATE DATABASE SalesDB;

USE SalesDB;

CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Quantity INT,
    Price INT
);

CREATE PROCEDURE AddSale(
    IN p_SaleID INT,
    IN p_ProductName VARCHAR(50),
    IN p_Quantity INT,
    IN p_Price INT

```

```

)
BEGIN
    INSERT INTO Sales
    VALUES (p_SaleID, p_ProductName, p_Quantity, p_Price);
END //
DELIMITER ;
CALL AddSale(1, 'Laptop', 2, 50000);
CALL AddSale(2, 'Mobile', 3, 20000);
CALL AddSale(3, 'Keyboard', 5, 1000);

SELECT * FROM Sales;
DELIMITER //
CREATE FUNCTION TotalSales()
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE Total INT;
    SELECT SUM(Quantity * Price)
    INTO Total
    FROM Sales;
    RETURN Total;
END //
DELIMITER ;
SELECT TotalSales() AS Total_Amount;

```

Q6) Triggers and Cursors

Develop an Employee Management System with Triggers for Audit Logging and Cursors for Efficient Data Processing Objective: Implement triggers and cursors in PL/SQL. Task: • Create a trigger on the Employees table that logs changes (INSERT, UPDATE, DELETE) into the Employee_Audit table. • Use a cursor to iterate through the Employees table and process salary increments based on certain conditions (e.g., increase salary by 10% for employees in the IT department)

```
CREATE DATABASE EmployeeDB;

USE EmployeeDB;

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary INT
);

CREATE TABLE Employee_Audit (
    AuditID INT AUTO_INCREMENT PRIMARY KEY,
    EmpID INT,
    ActionType VARCHAR(20),
    ActionDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO Employees VALUES
(1, 'Arjun', 'IT', 50000),
(2, 'Rahul', 'HR', 40000),
(3, 'Sneha', 'IT', 60000);

DELIMITER //

CREATE TRIGGER trg_insert
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit(EmpID, ActionType)
    VALUES (NEW.EmpID, 'INSERT');
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER trg_update
AFTER UPDATE ON Employees
FOR EACH ROW
BEGIN
```

```
INSERT INTO Employee_Audit(EmpID, ActionType)
VALUES (NEW.EmpID, 'UPDATE');
END //
```

```
DELIMITER ;
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_delete
AFTER DELETE ON Employees
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO Employee_Audit(EmpID, ActionType)
VALUES (OLD.EmpID, 'DELETE');
```

```
END //
```

```
DELIMITER ;
```

```
-- INSERT
```

```
INSERT INTO Employees VALUES
(4, 'Priya', 'Finance', 45000);
```

```
-- UPDATE
```

```
UPDATE Employees
```

```
SET Salary = 55000
```

```
WHERE EmpID = 1;
```

```
-- DELETE
```

```
DELETE FROM Employees
```

```
WHERE EmpID = 2;
```

```
SELECT * FROM Employee_Audit;
```

```
DELIMITER //
```

```
CREATE PROCEDURE IncreaseSalary()
```

```
BEGIN
```

```
DECLARE done INT DEFAULT 0;
```

```
DECLARE e_id INT;
```

```
DECLARE emp_cursor CURSOR FOR
```

```
SELECT EmpID
```

```

FROM Employees
WHERE Department = 'IT';  DECLARE CONTINUE HANDLER
FOR NOT FOUND SET done = 1;
OPEN emp_cursor;
read_loop: LOOP
    FETCH emp_cursor INTO e_id;
    IF done = 1 THEN
        LEAVE read_loop;
    END IF;
    UPDATE Employees
    SET Salary = Salary + (Salary * 0.10)
    WHERE EmpID = e_id;
END LOOP;
CLOSE emp_cursor;
END //
DELIMITER ;
CALL IncreaseSalary();
SELECT * FROM Employees;

```

Q7) NoSQL Database Introduction – MongoDB Aggregation and Indexing Objectives: Work with MongoDB for NoSQL data storage. Use MongoDB aggregation and indexing for data analysis. Task: • Create a MongoDB collection for user data. • Perform CRUD (Create, Read, Update, Delete) operations on the collection. • Write aggregation queries to calculate statistics (e.g., total sales, average age). • Create indexes and observe performance improvements.

```

UserDB
db.users.insertMany([
{
    name: "Arjun",
    age: 21,    city:
"Pune",    sales:
5000
},
{

```

```
    name: "Rahul",  
    age: 24,    city:  
    "Mumbai",  
    sales: 7000  
  },  
  {  
    name: "Sneha",  
    age: 20,  
    city: "Delhi",  
    sales: 4000  
  }  
])
```

```
//-----
```

```
// READ Operation
```

```
//-----
```

```
// Display all documents db.users.find()
```

```
// Find users with age greater than 21 db.users.find({  
age: { $gt: 21 } })
```

```
//-----
```

```
// UPDATE Operation
```

```
//-----
```

```
// Update sales of Arjun db.users.updateOne(  
  { name: "Arjun" },  
  { $set: { sales: 6000 } }  
)
```

```
// Display updated data db.users.find()
```

```

//-----
// DELETE Operation
//----- // Delete Rahul record db.users.deleteOne({ name: "Rahul" })

// Display data after delete db.users.find()

//-----
// Aggregation Queries
//-----

// Calculate total sales db.users.aggregate([
{
  $group: { _id: null,
totalSales: { $sum: "$sales" }
  }
}
])

// Calculate average age db.users.aggregate([
{
  $group: { _id: null,
averageAge: { $avg: "$age" }
  }
}
])

//----- // Create Index
//-----

// Create index on name db.users.createIndex({
name: 1 })

```

```
// Create index on age db.users.createIndex({  
age: 1 })
```

```
//-----  
// Check Query Performance  
//-----
```

```
db.users.find({ name: "Arjun" }).explain("executionStats")
```

How to Execute

Step 1

Open MongoDB shell.

Command: mongosh

Step 2

Paste commands one by one.

Step 3

Press Enter after every command.

Expected Output

find()

Displays all documents.

Aggregation

Shows:

totalSales averageAge

Q8) Case Study – Social Media Data Processing using NoSQL Objective: Process unstructured social media data using NoSQL databases. Task: • Import a dataset with social media posts into MongoDB.

- Write queries to filter and analyze data (e.g., find top trending hashtags).

```
// Create Database use SocialMediaDB
```

```
//-----
```

```
// Create Collection and Import Dataset
```

```
//-----
```

```
db.posts.insertMany([
{
  user: "Arjun",
  post: "Learning MongoDB is fun",
  likes: 120,
  hashtags: ["#MongoDB", "#NoSQL", "#Database"]
},
{
  user: "Rahul",  post:
"AI is the future",  likes:
200,
  hashtags: ["#AI", "#ML", "#Technology"]
},
{
  user: "Sneha",
  post: "Working on web development",
  likes: 150,
  hashtags: ["#WebDev", "#JavaScript", "#MongoDB"]
},
{  user:
"Priya",
  post: "Machine Learning project completed",
  likes: 180,  hashtags: ["#ML",
"#AI", "#Python"]
}
```

```
)
```

```
//-----
```

```
// Display All Data
```

```
//-----
```

```
db.posts.find()
```

```
//-----
```

```
// Filter Queries
```

```
//-----
```

```
// Find posts with likes greater than 150
```

```
db.posts.find({ likes: { $gt: 150 }
```

```
})
```

```
// Find posts containing #AI hashtag
```

```
db.posts.find({ hashtags: "#AI"
```

```
})
```

```
//-----
```

```
// Analyze Data using Aggregation
```

```
//-----
```

```
// Find total likes db.posts.aggregate([
```

```
{
```

```
  $group: { _id: null,
```

```
  totalLikes: { $sum: "$likes" }
```

```
  }
```

```
}
```

```
])
```

```
// Find average likes db.posts.aggregate([
```

```
{
  $group: {    _id: null,
averageLikes: { $avg: "$likes" }
  }
}
])
```

```
// Find top trending hashtags db.posts.aggregate([
```

```
{
  $unwind: "$hashtags"
},
{
  $group: {    _id:
"$hashtags",
count: { $sum: 1 }
  }
},
{
  $sort: { count: -1 }
}
])
```

```
//-----
```

```
// Create Index
```

```
//-----
```

```
db.posts.createIndex({ hashtags: 1 })
```

```
//-----
```

```
// Check Performance
```

```
//-----
```

```
db.posts.find({  
  hashtags: "#AI"  
}).explain("executionStats")\
```

How to Execute

Step 1

Open MongoDB shell:

mongosh

Step 2

Paste commands one by one.

Step 3

Press Enter after each command.

Expected Output

find()

Displays all social media posts.

Aggregation Displays:

```
totalLikes  
averageLikes top  
hashtags
```

Q9) MySQL Database Backup and Recovery Objective: To understand and practice the process of taking a backup of a MySQL database to ensure data security and disaster recovery

Step 1: Create Sample Database

```
CREATE DATABASE BackupDB;
```

```
USE BackupDB;
```

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

```
INSERT INTO Student VALUES  
(1, 'Arjun', 21),  
(2, 'Rahul', 22),  
(3, 'Sneha', 20);
```

```
SELECT * FROM Student;
```

Step 2: Take Backup of Database Open CMD

and run: `mysqldump -u root -p BackupDB >`

`backup.sql` Then:

- Enter MySQL password
- Backup file `backup.sql` will be created

Step 3: Check Backup File

You can see: `backup.sql`
inside your current folder.

Usually found in:

`C:\Users\YourName`

Step 4: Delete Database (For Recovery Demo)

Login MySQL: `mysql -u root -p` Then run:

`DROP DATABASE BackupDB;`

Step 5: Recover Database

Exit MySQL using: `exit`

Now run recovery command in CMD: `mysql`

`-u root -p BackupDB < backup.sql`

Step 6: Verify Recovery

Login again: `mysql -u`

`root -p`

Then:

`USE BackupDB;`

`SELECT * FROM Student;`

You will see restored data.